

Spring Day01

1.什么是Spring

spring是目前比较主流的Web框架，是Java世界上最为成功的框架。2004年发布的第一版，用于简化企业级应用程序的开发难度和周期。以IOC和AOP为核心。

IOC：Inverse Of Control，控制反转。指的是将对象的创建权交给Spring框架。在这之前都是由我们通过new关键字来创建的，而使用Spring之后，对象的创建都交给了Spring框架。

AOP：Aspect Oriented Programming，面向切面编程。用于封装多个类的公共行为，将那些与业务无关，却为模块所共同调用的逻辑封装起来，减少系统的重复代码。另外，AOP还可以解决一些系统层面上的问题，比如日志，事务，权限等。

2.Spring框架的特点

2.1 方便解耦，简化开发

Spring就是一个大工厂，可以将所有的对象的创建和依赖关系的维护交给spring管理。

2.2 方便集成各种优秀框架

Spring不排斥各种优秀的开源框架，其内部对何种优秀框架能够直接支持。如Struts2，Hibernate，MyBatis等。

2.3 降低Java EE API 的使用难度

Spring对JavaEE开发中非常难用的一些API（JDBC，JavaMail）等都提供了封装，是这些API应用的难度大大降低。

2.4 方便程序的测试

Spring支持JUnit，可以通过注解方便地测试Spring程序。

2.5 AOP 编程的支持

Spring 提供面向切面编程，可以方便地实现对程序进行权限拦截和运行监控等功能。

2.6 声明式事务的支持

只需要通过配置就可以完成对事务的管理，而无须手动编程

3.Spring的体系结构

Spring 框架采用分层的理念，根据功能的不同划分成了多个模块，这些模块大体可分为 Data Access/Integration（数据访问与集成）、Web、AOP、Aspects、Instrumentation（检测）、Messaging（消息处理）、Core Container（核心容器）和 Test。如下图所示（以下是 Spring Framework 4.x 版本后的系统架构图）

1. Data Access/Integration（数据访问／集成）

数据访问／集成层包括 JDBC、ORM、OXM、JMS 和 Transactions 模块，具体介绍如下。

JDBC 模块：提供了一个 JDBC 的样例模板，使用这些模板能消除传统冗长的 JDBC 编码还有必须的事务控制，而且能享受到 Spring 管理事务的好处。

ORM 模块：提供与流行的“对象-关系”映射框架无缝集成的 API，包括 JPA、JDO、Hibernate 和 MyBatis 等。而且还可以使用 Spring 事务管理，无需额外控制事务。

OXM 模块：提供了一个支持 Object /XML 映射的抽象层实现，如 JAXB、Castor、XMLBeans、JiBX 和 XStream。将 Java 对象映射成 XML 数据，或者将 XML 数据映射成 Java 对象。

JMS 模块：指 Java 消息服务，提供一套“消息生产者、消息消费者”模板用于更加简单的使用 JMS，JMS 用于用于在两个应用程序之间，或分布式系统中发送消息，进行异步通信。

Transactions 事务模块：支持编程和声明式事务管理。

2. Web模块

Spring 的 Web 层包括 Web、Servlet、WebSocket 和 Portlet 组件，具体介绍如下。

Web 模块：提供了基本的 Web 开发集成特性，例如多文件上传功能、使用的 Servlet 监听器的 IOC 容器初始化以及 Web 应用上下文。

Servlet 模块：提供了一个 Spring MVC Web 框架实现。Spring MVC 框架提供了基于注解的请求资源注入、更简单的数据绑定、数据验证等及一套非常易用的 JSP 标签，完全无缝与 Spring 其他技术协作。

WebSocket 模块：提供了简单的接口，用户只要实现响应的接口就可以快速的搭建 WebSocket Server，从而实现双向通讯。

Portlet 模块：提供了在 Portlet 环境中使用 MVC 实现，类似 Web-Servlet 模块的功能。

3. Core Container（Spring的核心容器）

Spring 的核心容器是其他模块建立的基础，由 Beans 模块、Core 核心模块、Context 上下文模块和 SpEL 表达式语言模块组成，没有这些核心容器，也不可能有 AOP、Web 等上层的功能。具体介绍如下。

Beans 模块：提供了框架的基础部分，包括控制反转和依赖注入。

Core 核心模块：封装了 Spring 框架的底层部分，包括资源访问、类型转换及一些常用工具类。

Context 上下文模块：建立在 Core 和 Beans 模块的基础之上，集成 Beans 模块功能并添加资源绑定、数据验证、国际化、Java EE 支持、容器生命周期、事件传播等。ApplicationContext 接口是上下文模块的焦点。

SpEL 模块：提供了强大的表达式语言支持，支持访问和修改属性值，方法调用，支持访问及修改数组、容器和索引器，命名变量，支持算数和逻辑运算，支持从 Spring 容器获取 Bean，它也支持列表投影、选择和一般的列表聚合等。

4. AOP、Aspects、Instrumentation和Messaging

在 Core Container 之上是 AOP、Aspects 等模块，具体介绍如下：

AOP 模块：提供了面向切面编程实现，提供比如日志记录、权限控制、性能统计等通用功能和业务逻辑分离的技术，并且能动态的把这些功能添加到需要的代码中，这样各司其职，降低业务逻辑和通用功能的耦合。

Aspects 模块：提供与 AspectJ 的集成，是一个功能强大且成熟的面向切面编程（AOP）框架。

Instrumentation 模块：提供了类工具的支持和类加载器的实现，可以在特定的应用服务器中使用。

messaging 模块：Spring 4.0 以后新增了消息（Spring-messaging）模块，该模块提供了对消息传递体系结构和协议的支持。

4.搭建Spring程序

搭建spring程序的时候需要如下几个Jar包

1.spring-core

2.spring-beans

3.spring-context

4.spring-expression

4.1 引入jar包

将上述的jar包引入到项目中并添加到classpath中

4.2 引入配置文件

applicationContext.xml , 放在classpath下。

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
                           http://www.springframework.org/schema/beans/spring-beans.xsd">

</beans>
```

5.Spring中的IOC和DI

IOC容器是Spring的核心，也可以称为Spring容器。Spring是通过IOC容器来管理对象的实例化和初始化，以及对象从创建到销毁的整个生命周期。

Spring 中使用的对象都由 IoC 容器管理，不需要我们手动使用 new 运算符创建对象。由 IoC 容器管理的对象称为 Spring Bean，Spring Bean 就是 Java 对象，和使用 new 运算符创建的对象没有区别。

Spring 通过读取 XML 或 Java 注解中的信息来获取哪些对象需要实例化。Spring 提供 2 种不同类型的 IoC 容器，即 BeanFactory 和 ApplicationContext 容器

5.1 BeanFactory容器

BeanFactory是最简单的容器，由org.springframework.beans.factory.BeanFactory接口定义，采用懒加载（lazy-load），所以容器启动的较快。BeanFactory提供了容器最基本的功能，配置文件加载时不会创建对象，在获取bean时才会创建对象。

BeanFactory就是一个管理Bean的工厂，它主要负责初始化各种Bean，并调用他们的生命周期方法。

使用BeanFactory，需要创建XmlBeanFactory的实例

（org.springframework.beans.factory.xml.XmlBeanFactory），通过XmlBeanFactory类的构造函数来传递Resource对象。

```
Resource resource = new ClassPathResource("applicationContext.html");
BeanFactory factory = new XmlBeanFactory(resource);
```

5.2 ApplicationContext容器

ApplicationContext集成了BeanFactory接口，由org.springframework.context.ApplicationContext接口定义，对象在启动容器时加载。ApplicationContext在BeanFactory的基础上增加了很多企业级功能。例如AOP，国际化，事件支持等。

ApplicationContext接口有两个常用的实现类，具体如下：

5.2.1 ClassPathXmlApplicationContext

该类从类路径ClassPath中寻找制定的XML配置文件，并完成ApplicationContext的实例化工作，具体如下。

```
ApplicationContext applicationContext = new ClassPathXmlApplicationContext(String
configLocation)
```

上述代码中，configLocation用于制定Spring配置文件的名称和位置。

5.2.2 FileSystemXmlApplicationContext

该类从指定的文件系统路径中寻找制定的XML配置文件，并完成ApplicationContext的实例化工作。

```
ApplicationContext applicationContext = new
FileSystemXmlApplicationContext(String configLocation)
```

在实际开发中，通常使用ApplicationContext，如果系统资源较少时，才考虑使用BeanFactory。

两者的区别在于，如果Bean的某一个属性没有注入，使用BeanFactory加载后，第一次调用getBean（）会抛出异常，而ApplicationContext则会在初始化的时候进行自检，这样有利于检查所依赖的属性是否注入。

5.3 Spring加载多个配置文件

1.使用字符串参数, 逗号分隔。

```
ApplicationContext ac = new  
ClassPathApplicationContext("config/a.xml", "configb.xml");
```

2.使用字符串数组

```
String[] config = {"config/a.xml", "config/b.xml"};  
ApplicationContext ac = ClassPathXmlApplicationContext(config);
```